Master in Artificial Intelligence and Data Science (IASD)
Module: Data Spaces & Data Integration & Semantic Interoperability

A Federated Mini Data Space for Health and Air Quality

Governance, signed classifications and privacy-limited aggregates in a three-node prototype (Tangier, fictitious data)

Final Practical Work (TP 1 – TP 4)

Students: Marwa Begdouri Terraf
Aya El Mansouri

Supervisor: Ouafae Baida

Academic year: 2026–2027

November 2026

Abstract

This report documents the design, implementation and validation of a prototype *federated data space* that lets a university laboratory study the association between PM2.5 air pollution and respiratory hospital admissions in Tangier, without centralising the underlying data. Two organisations keep their own SQLite databases: a regional environment directorate (node A) and a regional hospital (node B). A laboratory (node C) only orchestrates requests. The work was built incrementally in four practical sessions: local storage (TP1), open REST APIs with FastAPI (TP2), API-key protection and local computation at the hospital (TP3), and a governance layer (TP4) comprising a DCAT-inspired catalogue, a purpose-based contract (ODRL-inspired), HMAC-SHA256 signed classifications, a minimum publication threshold of three days, and a Laplace-noise mechanism with a per-contract privacy budget. On the fictitious ten-day dataset, the system yields mean admissions of 16.2 per day on days above the PM2.5 threshold (30 µg/m^3) and 7.8 per day below, a difference of 8.4. We show how TP4 closes the difference attack demonstrated at the end of TP3, and we discuss honestly why the resulting safeguards are pedagogical and do not constitute formal privacy guarantees. The report maps every requirement of the assignment to the implemented code, tests and evidence.

Keywords: data spaces, federated analysis, FastAPI, SQLite, HMAC, data governance, Laplace noise, difference attack.

Contents

Abstract **1**

1 Introduction **6**
1.1 Context . . . 6
1.2 Objectives . . . 6
1.3 Method and report organisation . . . 6

2 Requirements and Compliance **8**
2.1 Deliverables required by the assignment . . . 8
2.2 Technical requirements per TP . . . 8
2.3 Differences between the assignment code and our implementation . . . 10

3 System Architecture and Design **11**
3.1 Overall architecture . . . 11
3.2 Interaction protocol (TP4) . . . 12
3.3 Repository structure . . . 12
3.4 Technologies and tools . . . 13
3.5 Data model . . . 13

4 Implementation **14**
4.1 TP1 – Federated storage . . . 14
4.2 TP2 – Open REST APIs . . . 15
4.3 TP3 – Access control and local computation . . . 17
4.4 TP4 – Governance and privacy . . . 18
4.4.1 Configuration (node A) . . . 18
4.4.2 Catalogue and contract negotiation . . . 18
4.4.3 Signed classifications . . . 18
4.4.4 Controls at node B . . . 19
4.4.5 Minimum threshold and Laplace noise . . . 19
4.4.6 Laboratory orchestrator . . . 20

5 Testing and Validation **22**
5.1 Test strategy . . . 22
5.2 Manual tests via Swagger . . . 23
5.3 Negative tests . . . 24
5.4 Self-check script . . . 24

6 Results and Discussion **25**
6.1 Manual calculations (TP4) 25
6.2 Observed outputs 25
6.3 Discussion 28

7 Answers to the Questions of the TPs **29**

8 Limitations and Possible Improvements **31**

9 Conclusion and Perspectives **32**

List of Figures

3.1 Architecture of the prototype. Databases never leave their owners; only categories (A→C→B) and aggregates (B→C) circulate. 11
3.2 Sequence diagram of the TP4 scenario, as implemented in `client_analyse.py`. . 12

4.1 TP1: creation and content of the two local databases (five rows each). 14
4.2 TP2: Swagger interface of the environment API (port 8001), `GET /mesures` returning 200 with the five PM2.5 records. 15
4.3 TP2: Swagger interface of the hospital API (port 8002), `GET /admissions` returning 200 with the five admission records. 16
4.4 TP2: output of the laboratory client (means 16.0 and 8.5). 16
4.5 TP3: laboratory result (16.0, 8.5, difference 7.5). 17
4.6 TP3: difference attack reveals the exact admission count (8.0) of 1 October. . . . 18

5.1 Output of `python -m pytest tests -v`: 37 tests passed. 22
5.2 TP4: Swagger interface of node A listing `/health`, `/catalog`, `/contracts/negotiate`, `/classifications` and `/audit`. 23
5.3 TP4: Swagger interface of node B (`POST /compute`). 23

6.1 TP4 laboratory client without noise: catalogue, contract, results 16.2 / 7.8, and negative tests (403 purpose, 404 dataset, 401 key, 403 forged signature, 401 hospital key). 26
6.2 TP4 laboratory client with `-bruit` (first run). 27
6.3 TP4 laboratory client with `-bruit` (second run, different noisy means). 28

List of Tables

1.1 The four practical sessions and the question each one answers. 7

2.1 Final deliverables and where they are addressed. 8
2.2 Requirement-to-implementation mapping. 9
2.3 Deviations and additions relative to the assignment's reference code. 10

3.1 Nodes, roles and ports. 11
3.2 Technologies used (as declared in `requirements.txt` and the code). 13
3.3 Database schemas. 13

4.1 Order of checks at node B and resulting status codes. 19
4.2 Analytical noise level for a group of $n = 5$ days (derived from $b = (30/n)/\varepsilon$ and $\sigma = \sqrt{2}\,b$; not measured). 20

5.1 Automated tests defined in the repository (counted from the source files). 22
5.2 Negative and security tests, with expected status and implementing test. 24

6.1 TP4 dataset and resulting categories. 25

Chapter 1

Introduction

1.1 Context

A regional data space dedicated to health and air quality is being set up to allow several organisations to study the association between atmospheric pollution and respiratory admissions. Three actors take part: the regional environment directorate (organisation A), which holds daily PM2.5 measurements; the regional hospital (organisation B), which holds daily respiratory admissions; and a university laboratory (organisation C), which analyses authorised results but owns neither dataset. All data are fictitious.

The central question of the assignment is: *how can data held by different organisations be analysed together, without centralising them, while controlling their use and limiting what is revealed?*

1.2 Objectives

Following the assignment, the project aims to:

- understand federated storage and the retention of data at their owners;
- create and query two independent SQLite databases;
- expose data and results through REST APIs built with FastAPI;
- query separate services from a Python client;
- perform local computations and exchange only aggregated statistics;
- apply governance (purpose, contract) and discuss the limits of confidentiality.

1.3 Method and report organisation

The prototype was built step by step, TP1 to TP4, each step answering one question (Table 1.1). Chapter 2 lists the requirements and our compliance; Chapter 3 presents the architecture; Chapter 4 details the implementation; Chapter 5 describes tests and validation; Chapter 6 presents results; Chapter 7 answers the questions of each TP; Chapter 8 discusses limitations; the conclusion distinguishes federated storage, processing and analysis.

Table 1.1: The four practical sessions and the question each one answers.

TP	Question	What is built
1	Where is the data?	Two independent local databases
2	How do organisations communicate?	Two APIs and a client (raw data reach the laboratory)
3	How can raw data be avoided?	Local computation at the hospital, API key
4	How to control usage and limit disclosure?	Catalogue, contract, signature, minimum threshold, noise

Chapter 2

Requirements and Compliance

This chapter extracts the deliverables of the assignment ("Travail final à rendre") and the technical requirements of each TP, and maps them to our implementation. "Implemented" means that the source code is present in the repository; execution evidence (screenshots, terminal output) is referenced in Chapter 5.

2.1 Deliverables required by the assignment

Table 2.1: Final deliverables and where they are addressed.

Required deliverable	Where / how it is covered	Ref.
Python files of the four TPs	Repository: `organisation_A/`, `organisation_B/`, `laboratoire_C/`; inventory in Appendix ??	App. A
Generated SQLite databases	`environnement_tp1.db`, `environnement.db`, `hopital_tp1.db`, `hopital.db` created by the setup scripts (to be zipped with the submission)	§3.5
Screenshots: installation, `/docs`, results	Figures in Chapter 5 (placeholders until captured)	Ch. 5
Manual calculations of TP4	Section 6.1	§6.1
At least three negative tests (purpose, wrong key, forged signature)	Implemented and automated (Section 5.3); all three also present in the laboratory client	§5.3
Architecture diagram (organisations, databases, APIs, laboratory)	Figure 3.1	§3.1
Answers to the questions of each TP	Chapter 7	Ch. 7
Conclusion distinguishing federated storage / processing / analysis, and explaining how TP4 fixes TP3	Chapter 9	Ch. 9

2.2 Technical requirements per TP

Table 2.2: Requirement-to-implementation mapping.

TP	Requirement	Implementation (file)
1	Two distinct SQLite databases with the 5-day datasets	`setup_data.py -jeu tp1`, `setup_hopital.py -jeu tp1`
1	Display the content of both databases	`consulter_environnement.py`, `consulter_hopital.py`
2	One API per organisation (`/mesures`, `/statistiques`, `/admissions`)	`api_air_tp2.py`, `api_hopital_tp2.py`
2	Python client joining by date; expected means 16 and 8.5	`tp2_client.py`
3	A returns only categories, protected by API key (401 without/wrong key)	`api_air_v3.py`, `auth.py`
3	B computes locally from received categories; 400 on unknown category / duplicate dates	`api_hopital_v3.py`
3	Client: expected 16, 8.5, difference 7.5	`tp3_laboratoire.py`
3	Difference attack on a one-day group (returns 8.0)	`tp3_attaque_difference.py`; test `test_tp3_attaque_par_difference_reussit`
4	Step 0: 10-day databases, separate from TP1–3	`setup_data.py`, `setup_hopital.py`
4	Catalogue (`GET /catalog`, DCAT-inspired)	`api_air.py`
4	Contract negotiation with ordered checks 401 > 404 > 403 (purpose) > 403 (consumer)	`api_air.py`, `negocier_contrat`
4	Signed classifications (HMAC-SHA256) bound to contract and consumer	`signature.py`; endpoint `/classifications`
4	B checks: key (401), signature (403), content (400), budget (429)	`api_hopital.py`
4	Minimum publication threshold of 3 days	`confidentialite.py`, `resumer()`
4	Laplace noise, scale $(30/n)/\varepsilon$, budget per contract (4th call at ε=1 gives 429)	`confidentialite.py`; test `test_budget_epuise_au_4e_appel`
4	Client orchestrating the full scenario with negative tests; `-bruit` option	`client_analyse.py`

TP	Requirement	Implementation (file)
4	No `-reload`; contracts in memory	README, `api_air.py`
All	Answer discussion questions	Chapter 7

2.3 Differences between the assignment code and our implementation

Our code follows the behaviour prescribed by the assignment but is organised as Python packages rather than four flat scripts. The functional differences are listed below; none of them alters the expected status codes or results.

Table 2.3: Deviations and additions relative to the assignment's reference code.

Aspect	Our implementation
Code organisation	Packages `organisation_A` (api, database, security, `config.py`) and `organisation_B` (api, database, security, services) instead of flat files
Configuration	Keys, secret, purpose, threshold centralised in `config.py` (A); overridable by environment variables
API-key check at A	`cle_valide()` encodes both strings to bytes before `hmac.compare_digest` and rejects empty keys
Additional A endpoints	`GET /health`, `GET /audit` (access journal), and an audit trail recording every grant and refusal
Additional contract data	Contract stores `created_at` timestamp
Databases	TP1–3 and TP4 databases kept in separate files (`*_tp1.db` vs. `*.db`) so that they cannot overwrite each other
Services at B	Computation (`calcul_local.py`) and protection (`confidentialite.py`) separated from the HTTP layer
Client	`client_analyse.py` adds a seventh step (wrong hospital key → 401), a `-epsilon` option and environment-variable overrides; named `client_laboratoire.py` in the assignment
Tests	Automated pytest suites and a self-check script `verifier_A.py`, not required by the assignment

Chapter 3

System Architecture and Design

3.1 Overall architecture

The system comprises three independent nodes communicating only over HTTP/JSON (Figure 3.1). Node A exposes the catalogue, negotiates contracts and emits signed classifications; node B receives a signed packet, verifies it, and returns aggregates computed on its local database; node C is a Python client that never opens a SQLite file.

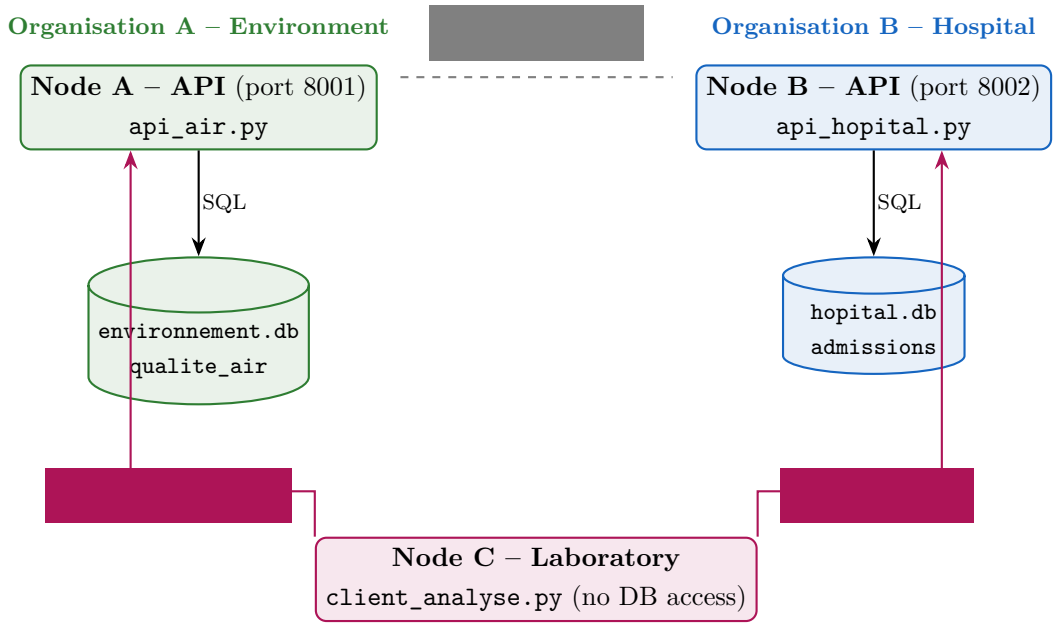


Figure 3.1: Architecture of the prototype. Databases never leave their owners; only categories (A→C→B) and aggregates (B→C) circulate.

Table 3.1: Nodes, roles and ports.

Node	Organisation	Role	Port
A	Environment directorate	PM2.5 data, catalogue, contract, signed classifications, audit	8001
B	Regional hospital	Admissions, local aggregates, minimum threshold, Laplace noise, budget	8002
C	University laboratory	Orchestrator; receives aggregates only	–

3.2 Interaction protocol (TP4)

The TP4 workflow has five stages (Figure 3.2): catalogue discovery, contract negotiation, retrieval of signed classifications, transmission of the signed packet to B, and local computation with return of aggregates only.

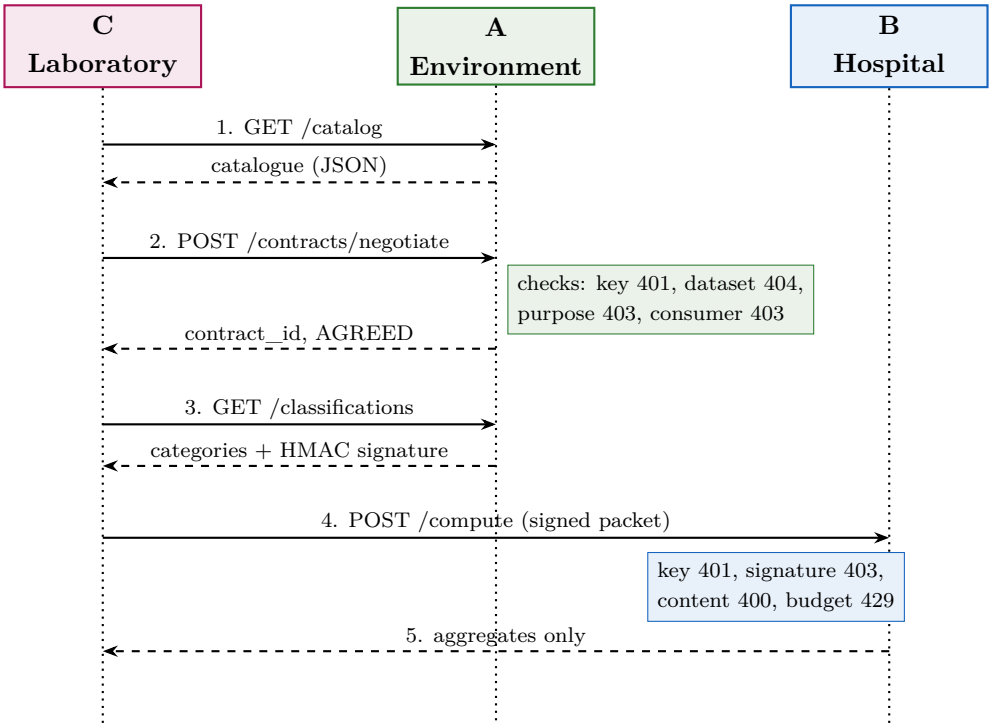


Figure 3.2: Sequence diagram of the TP4 scenario, as implemented in `client_analyse.py`.

3.3 Repository structure

```
mini-espace-federe/
|-- organisation_A/
|   |-- config.py               # keys, secret, purpose, threshold
|   |-- database/  setup_data.py, consulter_environnement.py
|   |-- api/       api_air_tp2.py, api_air_v3.py, api_air.py
|   |-- security/  auth.py, signature.py
|   `-- verifier_A.py           # self-check script
|-- organisation_B/
|   |-- database/  setup_hopital.py, consulter_hopital.py
|   |-- api/       api_hopital_tp2.py, api_hopital_v3.py, api_hopital.py
|   |-- services/  calcul_local.py, confidentialite.py
|   `-- security/  auth.py, signature.py
|-- laboratoire_C/ tp2_client.py, tp3_laboratoire.py,
|                  tp3_attaque_difference.py, client_analyse.py
|-- tests/         test_organisation_A.py, test_organisation_B.py,
|                  test_organisation_B_tp1_tp3.py
|-- requirements.txt, pytest.ini, README.md
```

Listing 3.1: Repository layout.

3.4 Technologies and tools

Table 3.2: Technologies used (as declared in `requirements.txt` and the code).

Tool	Role in the project
Python $\geq$ 3.10	Implementation language (type syntax such as `list[Classification]` is used)
SQLite (`sqlite3` module)	One local file-based database per organisation, no server
FastAPI	REST framework; automatic OpenAPI/Swagger documentation at `/docs`
Pydantic	Validation of request bodies (`DemandeContrat`, `RequeteCalcul`, ...)
Uvicorn	ASGI server running each node on its own port
Requests	HTTP client for laboratory scripts
Pytest, HTTPX	Automated tests (FastAPI `TestClient`)
Standard library: `hmac`, `hashlib`, `json`, `uuid`, `random`, `math`	Signature, contract identifiers, Laplace sampling
VS Code, virtual environment (`.venv`)	Development environment

3.5 Data model

Each organisation owns one table (Table 3.3). Constraints such as `CHECK(pm25 >= 0)` and `NOT NULL` were added in TP4 as in the assignment. Two datasets coexist: five days for TP1–3 and ten days for TP4, stored in separate files.

Table 3.3: Database schemas.

Database	Table	Columns
`environnement*.db`	`qualite_air`	`date TEXT PRIMARY KEY, ville TEXT NOT NULL, pm25 REAL NOT NULL CHECK(pm25>=0)`
`hopital*.db`	`admissions`	`date TEXT PRIMARY KEY, ville TEXT NOT NULL, nombre INTEGER NOT NULL CHECK(nombre>=0)`

Chapter 4

Implementation

4.1 TP1 – Federated storage

`setup_data.py` and `setup_hopital.py` create the databases using `contextlib.closing` so that connections are really closed (avoiding file locks on Windows) and the `-jeu tp1|tp4` option selects the 5-day or 10-day dataset. `consulter_environnement.py` and `consulter_hopital.py` print both tables. The two datasets live in distinct files, owned by distinct organisations: this is the basis of federated storage.

```
python organisation_A/database/setup_data.py --jeu tp1
python organisation_B/database/setup_hopital.py --jeu tp1
python organisation_A/database/consulter_environnement.py
python organisation_B/database/consulter_hopital.py
```

Listing 4.1: TP1 commands.

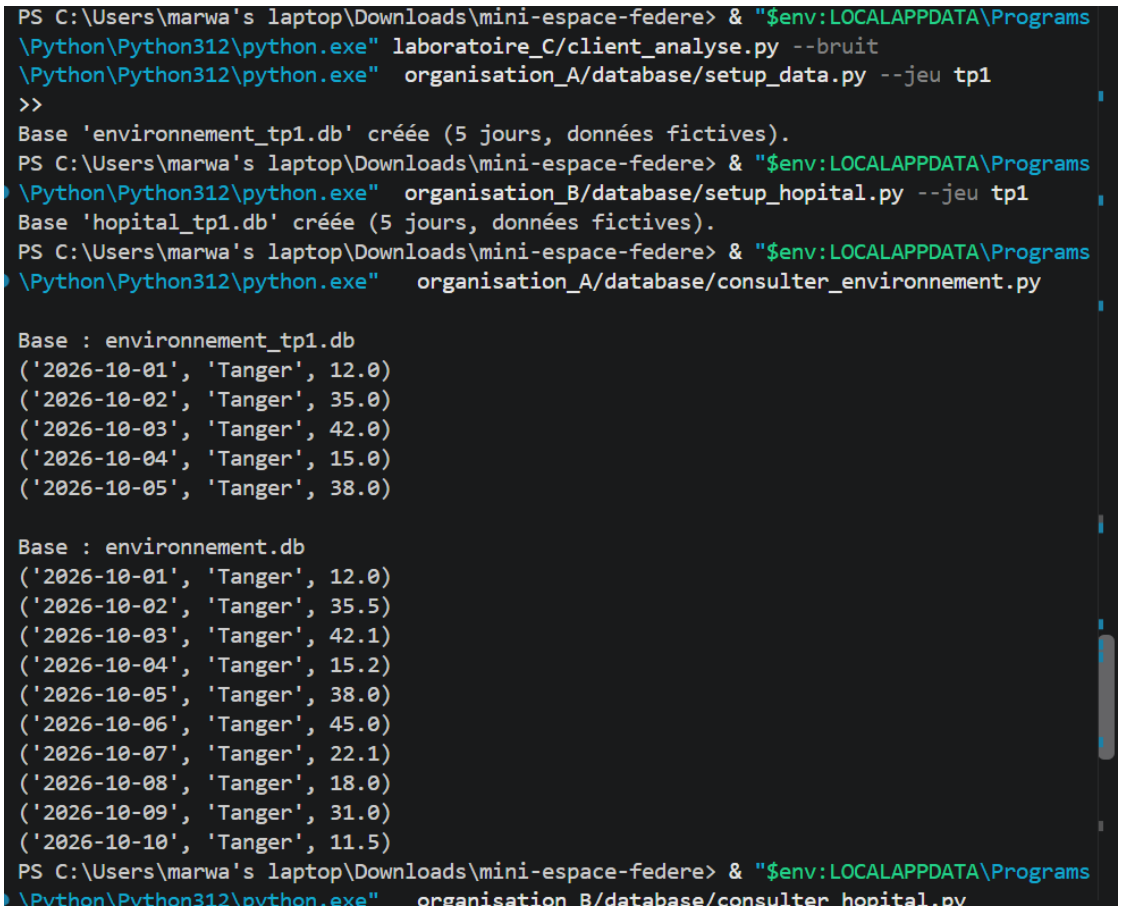


Figure 4.1: TP1: creation and content of the two local databases (five rows each).

4.2 TP2 – Open REST APIs

Each organisation exposes its raw data: node A via `/mesures` and `/statistiques`, node B via `/admissions` and `/statistiques`. No authentication is applied. The client `tp2_client.py` retrieves both datasets, joins them on the date, splits admissions according to PM2.5 > 30 and prints the two means. This version deliberately shows the problem to solve: the laboratory receives the *raw* PM2.5 values and admissions.

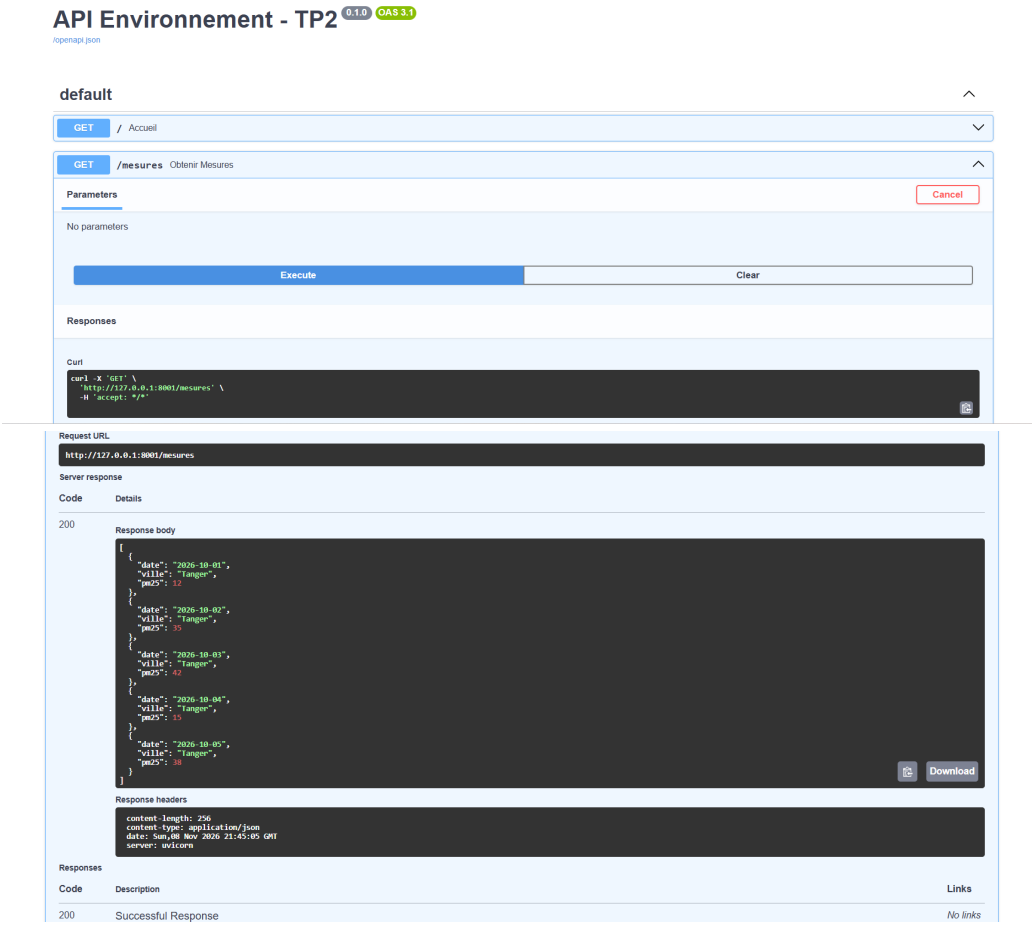


Figure 4.2: TP2: Swagger interface of the environment API (port 8001), `GET /mesures` returning 200 with the five PM2.5 records.

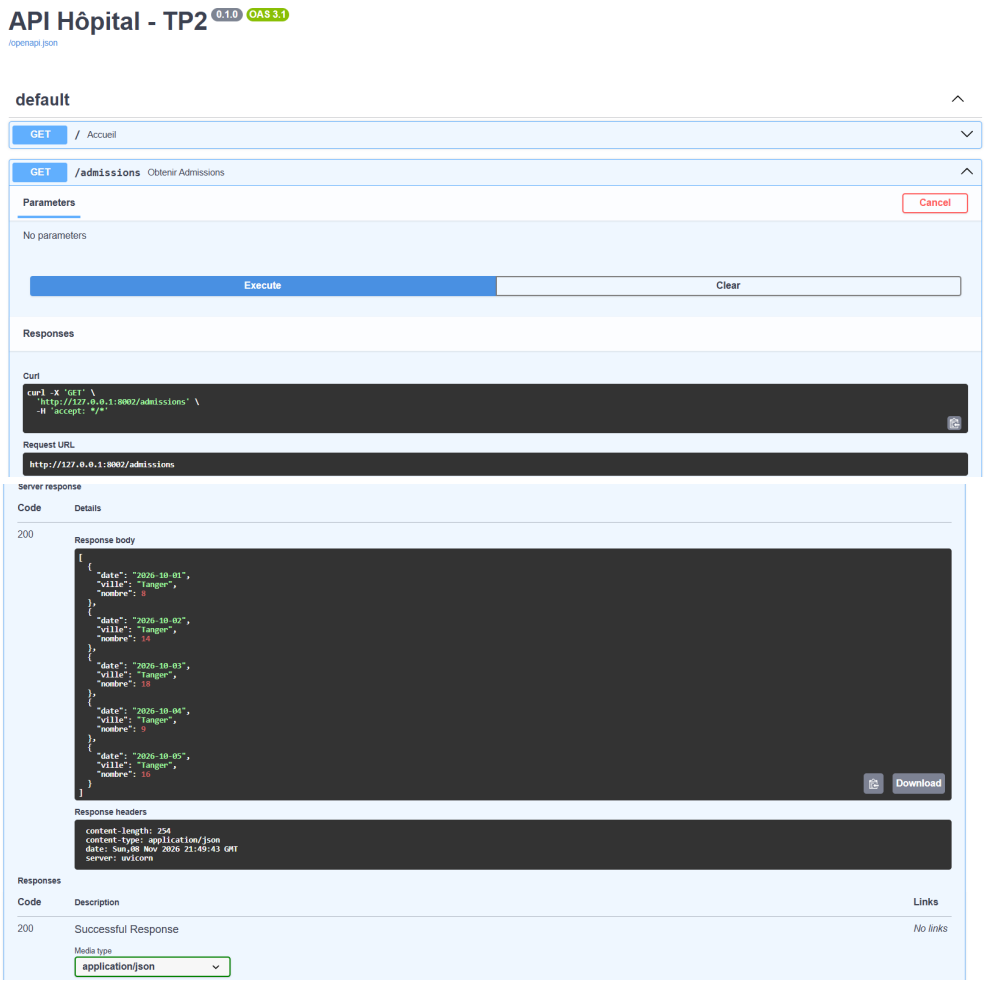


Figure 4.3: TP2: Swagger interface of the hospital API (port 8002), `GET /admissions` returning 200 with the five admission records.

```
PS C:\Users\marwa's laptop\Downloads\mini-espace-federe> & "$env:LOC
ALAPPDATA\Programs\Python\Python312\python.exe" laboratoire_C/tp2_cl
ient.py
Date | PM2.5 | Admissions
2026-10-01 | 12.0 | 8
2026-10-02 | 35.0 | 14
2026-10-03 | 42.0 | 18
2026-10-04 | 15.0 | 9
2026-10-05 | 38.0 | 16
Moyenne admissions, pollution élevée : 16.0
Moyenne admissions, autres jours : 8.5
PS C:\Users\marwa's laptop\Downloads\mini-espace-federe>
```

Figure 4.4: TP2: output of the laboratory client (means 16.0 and 8.5).

4.3 TP3 – Access control and local computation

Node A. `api_air_v3.py` exposes `GET /categories`. The key is passed in the `X-API-Key` header and verified by `cle_valide()`, which uses `hmac.compare_digest` (constant-time comparison) and rejects empty keys. The category is computed in SQL:

```
SELECT date,
       CASE WHEN pm25 > ? THEN 'elevee' ELSE 'non_elevee' END
FROM qualite_air ORDER BY date
```

Listing 4.2: Category computed at the source (api_air_v3.py).

Node B. `api_hopital_v3.py` exposes `POST /analyse-locale`. It checks the key (401), rejects categories outside {`elevee`, `non_elevee`} and duplicate dates (400), then joins the received categories with its local admissions and returns, for each category, the number of days and the mean.

Laboratory. `tp3_laboratoire.py` chains the two calls and prints the difference of means (expected 16, 8.5, difference 7.5).

Difference attack. Because the laboratory chooses the groups, sending a single date (`2026-10-01`, `non_elevee`) makes the hospital return a group of one day with mean `8.0`, i.e. the exact admission count of that day. The script `tp3_attaque_difference.py` reproduces this, and the test `test_tp3_attaque_par_difference_reussit` asserts it so that the weakness is documented, not hidden.

```
PS C:\Users\marwa's laptop\Downloads\mini-espace-federe> Set-Alias p
ython "$env:LOCALAPPDATA\Programs\Python\Python312\python.exe"
>> $env:CLE_ENVIRONNEMENT="demo-air-123"
>> $env:CLE_HOPITAL="demo-hopital-456"
>> python laboratoire_C/tp3_laboratoire.py
--- Analyse régionale ---
Pollution élevée : {'nombre_jours': 3, 'moyenne_admissions': 16.0}
Pollution non élevée : {'nombre_jours': 2, 'moyenne_admissions': 8.5
}
Différence entre les moyennes : 7.5
PS C:\Users\marwa's laptop\Downloads\mini-espace-federe>
```

Figure 4.5: TP3: laboratory result (16.0, 8.5, difference 7.5).

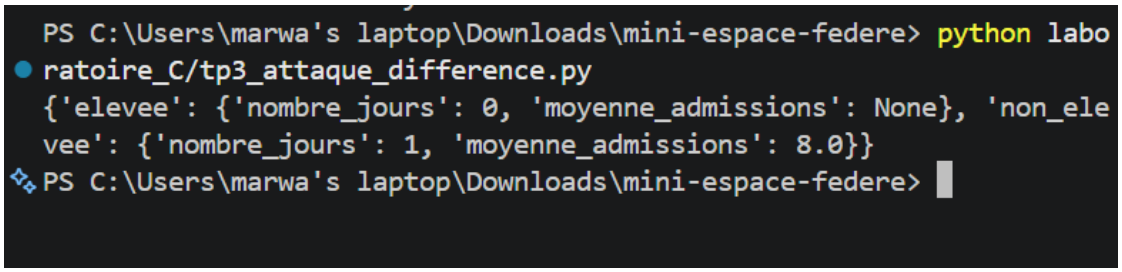


Figure 4.6: TP3: difference attack reveals the exact admission count (8.0) of 1 October.

4.4 TP4 – Governance and privacy

4.4.1 Configuration (node A)

`config.py` centralises the values that must be shared or tuned: dataset identifier `dataset-air-tanger`, authorised purpose `research`, authorised consumer `laboratoire-iasd`, threshold `SEUIL_PM25 = 30.0`, API key `CLE_AIR` and federation secret `SECRET_FEDERATION`, all overridable by environment variables. The demonstration defaults must never be used in production.

4.4.2 Catalogue and contract negotiation

`GET /catalog` returns a simplified, DCAT-inspired description of the dataset (identifier, title, description, publisher, temporal coverage, unit, access endpoint, `allowed_purpose`). The `allowed_purpose` field plays the role of an ODRL-inspired usage rule. `POST /contracts/negotiate` applies the checks in the order prescribed by the assignment:

1. API key → 401; 2. dataset exists → 404; 3. purpose is `research` → 403; 4. consumer authorised → 403.

If all checks pass, a contract `contract-air-<8 hex>` with status `AGREED` is stored *in memory* (hence the instruction not to use `-reload`). Every decision, accepted or refused, is appended to an in-memory journal exposed by the key-protected `GET /audit`; this audit trail is an addition to the assignment.

4.4.3 Signed classifications

`GET /classifications` re-checks key, contract existence, status/dataset, and that purpose and consumer match the contract (all refused with 403). It then converts each day to `au_dessus_seuil` or `sous_seuil`; **the PM2.5 value never leaves node A**. The list is signed by HMAC-SHA256 over a canonical JSON message

$$\sigma = \text{HMAC-SHA256}_K\left(\text{JSON}(\{\text{contract_id}, \text{consumer_id}, \text{classifications}\})\right), \tag{4.1}$$

where K is `SECRET_FEDERATION` and the JSON is serialised with sorted keys, compact separators `(",",":")`, `ensure_ascii=False` and UTF-8 encoding. Both nodes must use exactly the same serialisation, otherwise B returns 403 (this interface contract is stated in `README_A.md`).

```
1  def message_canonique(contract_id, consumer_id, classifications):
```

```
    return json.dumps(
        {"contract_id": contract_id, "consumer_id": consumer_id,
         "classifications": classifications},
        sort_keys=True, separators=(",", ":"), ensure_ascii=False,
    ).encode("utf-8")

def signer(contract_id, consumer_id, classifications, secret=None):
    secret = config.SECRET_FEDERATION if secret is None else secret
    msg = message_canonique(contract_id, consumer_id, classifications)
    return hmac.new(secret, msg, hashlib.sha256).hexdigest()
```

Listing 4.3: Canonical message and signature (organisation_A/security/signature.py).

4.4.4 Controls at node B

`POST /compute` (`api_hopital.py`) chains the checks in the following order:

Table 4.1: Order of checks at node B and resulting status codes.

Step	Status	Condition
1	401	API key missing or incorrect
2	403	HMAC signature does not match the received (contract, consumer, classifications)
3	400	Empty list; duplicate date; unknown category; date absent from hospital database
4	429	Noise requested (`apply_dp`) and privacy budget exhausted for this contract
5	200	Local computation; only aggregates are returned

Signature verification is what neutralises the TP3 attack: any sub-list, modified category or different consumer changes the canonical message, so the signature no longer matches and B answers 403 before any computation.

4.4.5 Minimum threshold and Laplace noise

`confidentialite.py` implements two output protections.

Minimum publication threshold. If a group contains fewer than `SEUIL_MIN_JOURS = 3` days, `resumer()` returns `moyenne = None` and status `SUPPRIME_GROUPE_TROP_PETIT`. This is *not* k-anonymity.

Laplace mechanism. When `apply_dp` is true, noise is added to the mean of a group of size n. With a pedagogical bound of 30 admissions per day, the sensitivity of the mean is $\Delta = 30/n$ and the Laplace scale is $b = \Delta/\varepsilon$. A sample is drawn by inverse transform from $u \in (-\frac{1}{2}, \frac{1}{2})$:

$$X = -b\,\mathrm{sgn}(u)\,\ln(1 - 2|u|), \qquad \sigma_X = \sqrt{2}\,b. \tag{4.2}$$

The result is clipped to $[0, 30]$ (post-processing) and the randomness comes from `random.SystemRandom` rather than the predictable default generator. Group sizes n are not noised.

```python
def bruit_laplace(echelle):
    u = RNG.random() - 0.5
    while u == -0.5:          # avoid log(0)
        u = RNG.random() - 0.5
    return -echelle * math.copysign(1.0, u) * math.log(1 - 2 * abs(u))

def resumer(valeurs, apply_dp, epsilon):
    n = len(valeurs)
    if n < SEUIL_MIN_JOURS:
        return {"n": n, "moyenne": None, "statut": "SUPPRIME_GROUPE_TROP_PETIT"}
    moyenne = sum(valeurs) / n
    if apply_dp:
        moyenne += bruit_laplace((BORNE_MAX / n) / epsilon)
        moyenne = min(max(moyenne, 0.0), BORNE_MAX)
    return {"n": n, "moyenne": round(moyenne, 2), "statut": "OK"}
```

Listing 4.4: Laplace sampling and summary (organisation_B/services/confidentialite.py).

Privacy budget. A cumulative budget `BUDGET_EPSILON = 3.0` is tracked per `contract_id` in memory; a request is refused with 429 if used $+ \varepsilon > 3$. With $\varepsilon = 1$ the fourth noisy call is therefore rejected.

Table 4.2: Analytical noise level for a group of $n = 5$ days (derived from $b = (30/n)/\varepsilon$ and $\sigma = \sqrt{2}\, b$; not measured).

ε	Scale b	Standard deviation σ
0.5	12	16.97
1.0	6	8.49
3.0	2	2.83

At $\varepsilon = 1$, $\sigma \approx 8.5$ is of the same order as the observed difference of means (8.4), which explains the strong variability of noisy results.

4.4.6 Laboratory orchestrator

`client_analyse.py` runs the full scenario: catalogue (step 1); three negative tests—purpose `commercial` (403), unknown dataset (404), wrong key (401) (step 2); contract (step 3); classifications (step 4); computation at B with or without noise via `-bruit [-epsilon x]` (step 5); a forged packet containing only the first four classifications (403, step 6); and a wrong hospital key (401, step 7). The client never accesses a database file.

```
python organisation_A/database/setup_data.py
python organisation_B/database/setup_hopital.py
python -m uvicorn organisation_A.api.api_air:app --port 8001      # terminal 1
python -m uvicorn organisation_B.api.api_hopital:app --port 8002  # terminal 2
```

```
python laboratoire_C/client_analyse.py                              # terminal 3
python laboratoire_C/client_analyse.py --bruit
```

Listing 4.5: TP4 launch sequence (three terminals, from the repository root, without `-reload`).

Chapter 5

Testing and Validation

5.1 Test strategy

Validation combines three levels: (i) manual tests through Swagger `/docs`, as prescribed by the assignment; (ii) scripted scenarios in the laboratory client; (iii) automated pytest suites. The node A tests use a temporary database and clear the in-memory state in a fixture; the node B tests simulate A by calling the same signature function. Table 5.1 summarises the test functions present in the repository.

Table 5.1: Automated tests defined in the repository (counted from the source files).

File	Tests	Coverage
`test_organisation_A.py`	15	Catalogue, negotiation, 8 negative cases, 10 signed days, no raw PM2.5, forged signature, signature bound to consumer, audit
`test_organisation_B.py`	13	Means 16.2/7.8, 401, 403 (subset and modified category), 400 cases, threshold, bounded and variable noise, budget (429), budget per contract
`test_organisation_B_tp1_tp3.py`	9	5-row database, TP2 endpoints, TP3 access control and results, TP3 difference attack
Total	**37**	

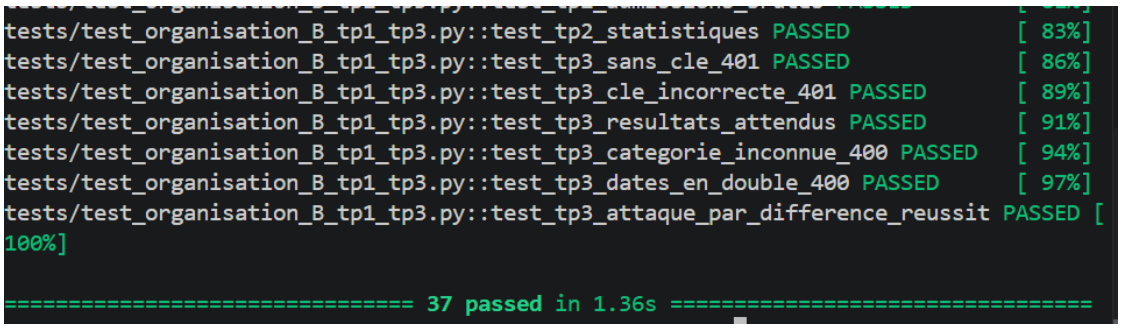


Figure 5.1: Output of `python -m pytest tests -v`: 37 tests passed.

All 37 automated tests passed (`37 passed in 1.36s`, Figure 5.1), which matches the number of test functions counted in Table 5.1.

5.2 Manual tests via Swagger

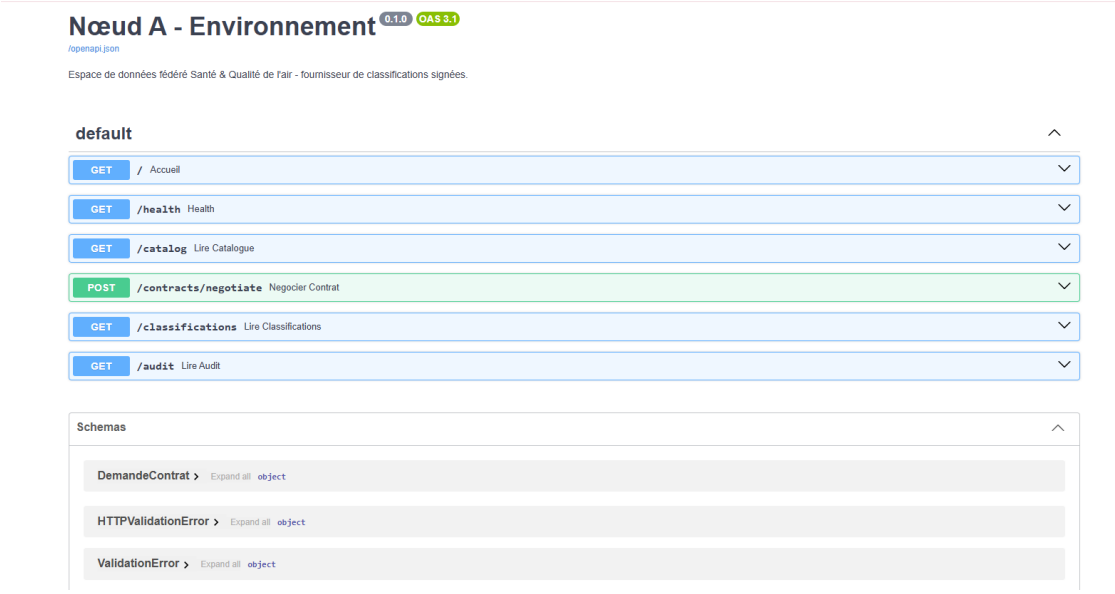


Figure 5.2: TP4: Swagger interface of node A listing `/health`, `/catalog`, `/contracts/negotiate`, `/classifications` and `/audit`.

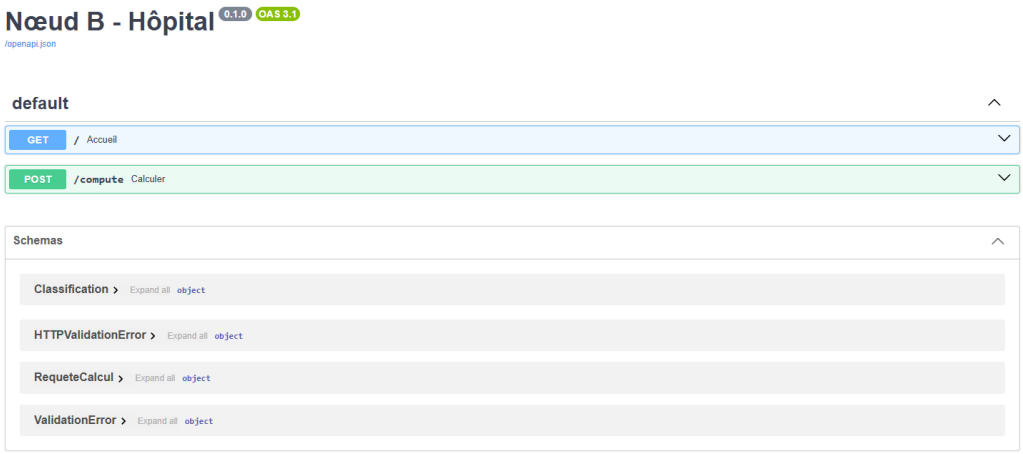


Figure 5.3: TP4: Swagger interface of node B (`POST /compute`).

5.3 Negative tests

Table 5.2: Negative and security tests, with expected status and implementing test.

Scenario	Status	Automated test / script
Purpose `commercial` (A)	403	`test_negatif_finalite_commerciale_403`; client step 2
Unknown dataset (A)	404	`test_negatif_dataset_inconnu_404`; client step 2
Wrong / missing API key (A)	401	`test_negatif_cle_incorrecte_401`, `test_negatif_sans_cle_401`; client step 2
Unknown consumer (A)	403	`test_negatif_consommateur_inconnu_403`
Unknown contract (A)	403	`test_negatif_contrat_inconnu_403`
Consumer different from contract (A)	403	`test_negatif_consommateur_different_du_contrat_403`
Wrong / missing API key (B)	401	`test_cle_incorrecte`, `test_cle_absente`; client step 7
Forged signature: sub-list (B)	403	`test_signature_falsifiee_sous_ensemble`; client step 6
Forged signature: category changed (B)	403	`test_signature_falsifiee_categorie_modifiee`
Unknown category / duplicate date / empty list / date not in DB (B)	400	four dedicated tests
Group of fewer than 3 days (B)	200	status `SUPPRIME_GROUPE_TROP_PETIT` (`test_seuil_minimal_...`)
Fourth noisy call at ε=1 (B)	429	`test_budget_epuise_au_4e_appel`

5.4 Self-check script

`verifier_A.py` replays what the laboratory will do against node A (catalogue, negotiation with 4 negative cases, classification structure, signature validity, truncated-list rejection, unknown contract, wrong consumer) and prints `PASS`/`FAIL` lines. It was written to validate the interface contract with the partner node before integration.

Chapter 6

Results and Discussion

6.1 Manual calculations (TP4)

Table 6.1 gives the ten-day dataset and the category assigned by node A (threshold PM2.5 > 30; 31.0 is above, 30.0 would be below).

Table 6.1: TP4 dataset and resulting categories.

Date (2026)	PM2.5 (µg/m^3)	Admissions	Category
01/10	12.0	8	`sous_seuil`
02/10	35.5	14	`au_dessus_seuil`
03/10	42.1	18	`au_dessus_seuil`
04/10	15.2	9	`sous_seuil`
05/10	38.0	16	`au_dessus_seuil`
06/10	45.0	20	`au_dessus_seuil`
07/10	22.1	10	`sous_seuil`
08/10	18.0	7	`sous_seuil`
09/10	31.0	13	`au_dessus_seuil`
10/10	11.5	5	`sous_seuil`

$$\bar{x}_{\text{above}} = \frac{14+18+16+20+13}{5} = \frac{81}{5} = 16.2$$
$$\bar{x}_{\text{below}} = \frac{8+9+10+7+5}{5} = \frac{39}{5} = 7.8$$
$$\Delta = 16.2 - 7.8 = 8.4 \text{ admissions per day.}$$

These values are the ones asserted by `test_calcul_sans_bruit` (`16.2` and `7.8`, $n = 5$ each). For the 5-day TP1–3 data, the same method gives $\frac{14+18+16}{3} = 16$ and $\frac{8+9}{2} = 8.5$, a difference of 7.5.

6.2 Observed outputs

Figure 6.1 shows the complete scenario run without noise. The three negative tests at node A return the expected 403 (purpose `commercial`), 404 (unknown dataset) and 401 (wrong key); the authorised request yields contract `contract-air-3ef35979` and ten classified days; node B returns means 16.2 and 7.8 (difference 8.40); the forged packet is refused with 403 and the wrong

hospital key with 401.

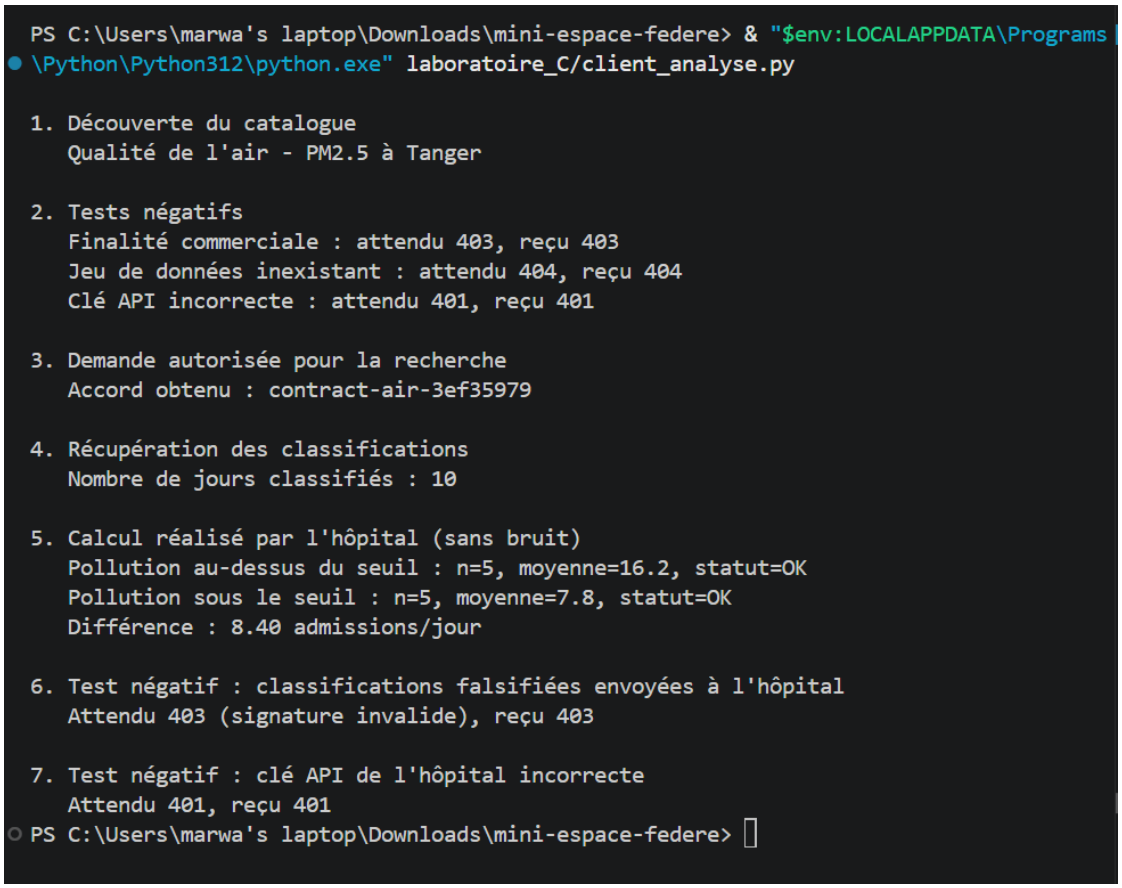


Figure 6.1: TP4 laboratory client without noise: catalogue, contract, results 16.2 / 7.8, and negative tests (403 purpose, 404 dataset, 401 key, 403 forged signature, 401 hospital key).

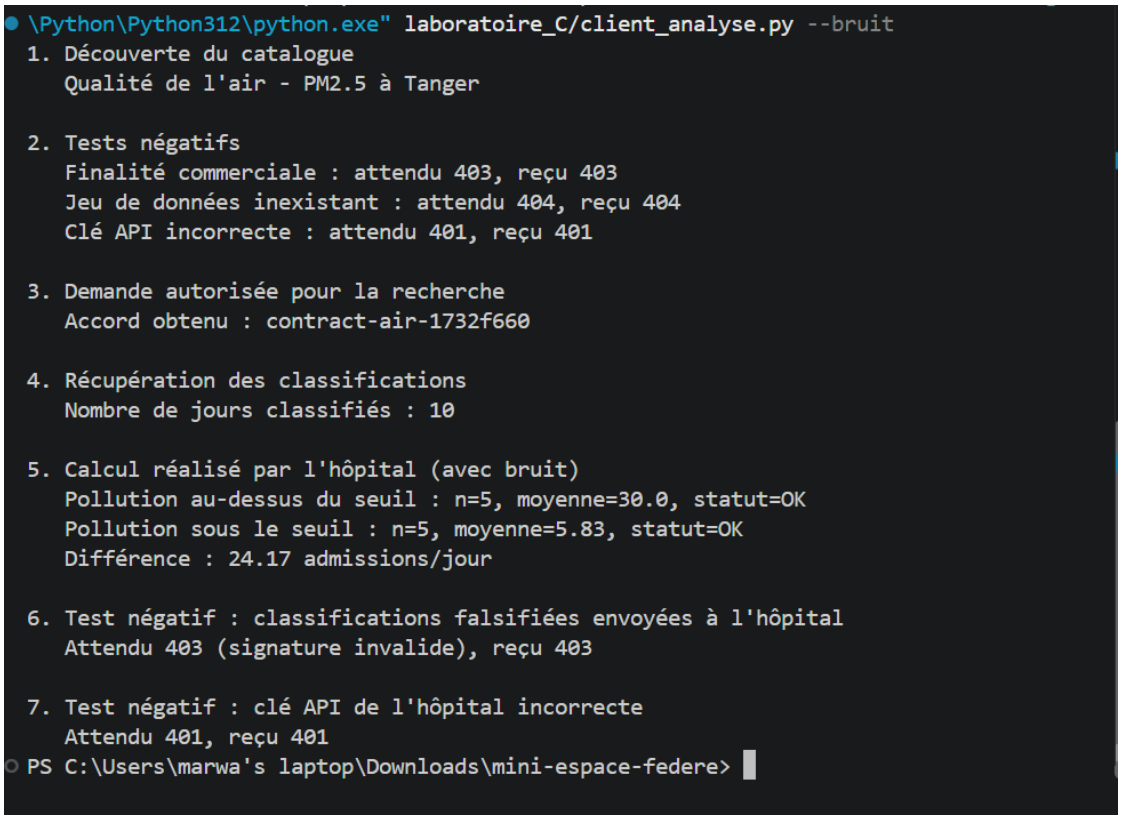


Figure 6.2: TP4 laboratory client with `-bruit` (first run).

In the noisy run of Figure 6.2 (contract `contract-air-1732f660`), the two group sizes remain $n = 5$, but the means became 30.0 for days above the threshold (exact value 16.2) and 5.83 for days below (exact value 7.8), giving an apparent difference of 24.17 instead of 8.40. The value 30.0 is the upper bound `BORNE_MAX`: the noise pushed the mean beyond 30 and the post-processing clipped it. This single run illustrates concretely the precision/protection trade-off discussed below: at $\varepsilon = 1$ the noisy result is far from the true difference.

```
PS C:\Users\marwa's laptop\Downloads\mini-espace-federe> & "$env:LOCALAPPDATA\Programs
\Python\Python312\python.exe" laboratoire_C/client_analyse.py --bruit
1. Découverte du catalogue
   Qualité de l'air - PM2.5 à Tanger

2. Tests négatifs
   Finalité commerciale : attendu 403, reçu 403
   Jeu de données inexistant : attendu 404, reçu 404
   Clé API incorrecte : attendu 401, reçu 401

3. Demande autorisée pour la recherche
   Accord obtenu : contract-air-7782bb24

4. Récupération des classifications
   Nombre de jours classifiés : 10

5. Calcul réalisé par l'hôpital (avec bruit)
   Pollution au-dessus du seuil : n=5, moyenne=5.96, statut=OK
   Pollution sous le seuil : n=5, moyenne=13.73, statut=OK
   Différence : -7.77 admissions/jour

6. Test négatif : classifications falsifiées envoyées à l'hôpital
   Attendu 403 (signature invalide), reçu 403

7. Test négatif : clé API de l'hôpital incorrecte
   Attendu 401, reçu 401
PS C:\Users\marwa's laptop\Downloads\mini-espace-federe>
```

Figure 6.3: TP4 laboratory client with `-bruit` (second run, different noisy means).

In the second noisy run (Figure 6.3, contract `contract-air-7782bb24`) the means were 5.96 above the threshold and 13.73 below, i.e. an apparent difference of -7.77: the sign of the true difference ($+8.40$) is reversed. Two executions on identical data therefore give very different, even contradictory, results, which is the intended effect of the noise and shows why $\varepsilon = 1$ is very protective but nearly useless for a group of five days.

6.3 Discussion

TP2 vs. TP3 vs. TP4. In TP2 the laboratory sees raw PM2.5 values and admissions. In TP3 it sees only categories from A and aggregates from B, but it can choose any date group. In TP4 it can only submit the complete list that A signed for its contract; B publishes only groups of at least three days and may add noise.

Precision versus protection. Noise at $\varepsilon = 1$ ($\sigma \approx 8.5$) can mask a difference of 8.4: stronger protection (smaller ε) degrades utility, and vice versa.

No causal claim. The difference suggests an association on a small fictitious sample; it does not establish causality.

Chapter 7

Answers to the Questions of the TPs

TP1

1. *Where are air-quality measurements stored?* In the SQLite file `environnement.db` (table `qualite_air`) owned by organisation A.
2. *Where are hospital admissions stored?* In `hopital.db` (table `admissions`) owned by organisation B.
3. *Same database?* No: two independent files, with different schemas and owners.
4. *How can the laboratory discover the available data without centralising files?* Through metadata published by each owner—a catalogue exposed by an API (implemented in TP4 as `GET /catalog`)—so that the laboratory learns what exists, its coverage and its usage conditions without copying the data.

TP2

1. FastAPI defines the REST endpoints, validates inputs and generates the OpenAPI/Swagger documentation.
2. Uvicorn is the ASGI server that runs the FastAPI application and handles HTTP connections.
3. Two independent services run on one machine; each needs its own port (8001 for A, 8002 for B) to avoid conflicts.
4. Yes. The laboratory receives raw PM2.5 values and raw admission counts.
5. No. REST is an architectural style; confidentiality requires authentication, authorisation, encryption in transit and data minimisation.

TP3

1. In TP2 the laboratory receives raw measurements and raw admissions; in TP3 it receives categories from A and aggregates (counts and means) from B.
2. To keep the raw admissions at their owner and send only results; computation moves to the data rather than data to the computation.
3. The hospital still receives the dates and the daily category (above/below threshold), which is coarse information about air quality on each date.

4. No. In the section 8 attack, a one-day group returns a mean of 8.0, i.e. the exact admission count of that day. An aggregate over a very small group is an individual value.
5. No. A key authenticates a caller but expresses neither purpose, contract nor limits on what may be requested; it can be shared or leaked and does not give the owner control after the data leave.
6. By defining the allowed operations server-side: signed requests that cannot be altered, a minimum group size, validation of payloads, contracts bound to a purpose, and a budget on repeated queries.
7. Identity and trust management for organisations, a federated catalogue, machine-readable policies and contracts, usage control, auditing/logging, encrypted transport (HTTPS), secure secret management and a risk analysis.

TP4

1. The catalogue fields: `dataset_id`, title, description, publisher, temporal coverage, unit, access endpoint and allowed purpose.
2. DCAT is the W3C vocabulary for describing data catalogues, datasets and their distributions [2]. Ours is a plain JSON structure with ad hoc field names, no RDF, no distributions or media types: it is inspired by DCAT but not conformant.
3. Purpose control ensures data are used only for the authorised aim. A catalogue *describes* datasets; a usage policy *states* what is permitted (ODRL-inspired `allowed_purpose` [3]); an access control *enforces* who may call what (API key, consumer check, contract).
4. Between nodes circulate the catalogue, contract identifiers, categories with dates, the signature, and aggregates. The raw PM2.5 values and admissions stay in their databases. Local retention is insufficient because the outputs themselves (categories, aggregates, possibly with small groups) can reveal individual values, as the TP3 attack shows.
5. A statistic over a very small group reveals individual values; a threshold of three days is a rule of thumb and not a k-anonymity guarantee [6], since it does not consider quasi-identifiers or combinations of queries. The signature restricts the laboratory to the complete list issued by A: any subset or modification is detected by B (403), which blocks the one-day group of the TP3 attack.
6. Without noise the means are 16.2 and 7.8; with noise they differ at each call and may even reverse the ordering, since $\sigma \approx 8.5$ at $\varepsilon = 1$ for $n = 5$. More noise gives more protection and less accuracy. The code is not a complete differential-privacy implementation: bounds are pedagogical, the budget is simplistic and in memory, there is no formal composition accounting and sampling uses floating point [5].
7. No. Association is not causation; one should consider temperature, humidity, season, influenza or other epidemics, day of week, holidays, pollen, delayed effects (lags), population and hospital capacity, and use more data and appropriate statistical models.

Chapter 8

Limitations and Possible Improvements

We list limitations that follow from the code and from the assignment.

- **Not a formal privacy guarantee.** The minimum threshold is not k-anonymity; the Laplace mechanism is an illustration, with a pedagogical bound of 30 and floating-point sampling.
- **Budget keyed by contract.** The budget is stored per `contract_id` in memory. Since a contract can be negotiated repeatedly without limit, a malicious consumer could obtain new contracts and repeat noisy queries to average the noise out. A budget per consumer and dataset, persisted, would close this gap.
- **Noise is opt-in.** The consumer decides `apply_dp`; a production system would enforce the policy server-side.
- **State in memory.** Contracts, journal and budgets are lost on restart, which is why `-reload` is forbidden.
- **Symmetric signature.** A and B share one HMAC secret, so B could also forge a signature, and any holder of the secret can sign. Asymmetric signatures (e.g. Ed25519) and per-organisation keys would provide non-repudiation.
- **No expiry or revocation.** A signed packet remains valid indefinitely; B does not query A to check that a contract is still active (replay is possible).
- **Static keys, no TLS.** Demonstration API keys, plain HTTP, no organisational identity.
- **Information still exchanged.** Daily categories and aggregates circulate: federation does not mean that no data are exchanged.
- **Scale and data.** Ten fictitious days; no statistical test, no confidence interval.

Possible improvements: persistent stores for contracts/budgets, expiry timestamps and nonces in the signed message, asymmetric signatures, TLS, a DCAT-conformant (JSON-LD) catalogue and ODRL policies, and a certified differential-privacy library.

Chapter 9

Conclusion and Perspectives

We built a three-node prototype in which each organisation keeps its data, the laboratory orchestrates the analysis, and only controlled outputs circulate. The three notions the assignment asks to distinguish are realised at different stages:

- **Federated storage** (TP1): data physically remain in separate databases owned by A and B; nothing is copied to a central repository.
- **Federated processing** (TP2–TP3): computations are executed by the node that owns the data. In TP2 this is only partial, since raw data still reach the laboratory; in TP3 the hospital computes locally and returns aggregates.
- **Federated analysis** (TP4): a result is obtained by combining information from several organisations through a governed workflow—catalogue, purpose-based contract, signed classifications from A, local aggregate computation at B—so that the analysis is jointly produced without central pooling of the raw data.

TP4 corrects the three weaknesses left by TP3: (i) the *free choice of groups* is removed because B accepts only the complete list signed by A for a given contract and consumer, and groups below three days are suppressed; (ii) the *lack of purpose control* is addressed by the contract negotiation that checks dataset, purpose and consumer before any classification is issued, and by the audit journal; (iii) the *lack of discovery* is addressed by the catalogue. Optional Laplace noise with a budget further limits what repeated queries reveal. These mechanisms remain pedagogical: stronger identities, persistent state, formal privacy accounting and transport security are required for a real data space.

Bibliography

[1] Master IASD, *Support de travaux pratiques – Mini-espace de données fédéré: Santé et qualité de l'air*, TP1–TP4, course handout.

[2] W3C, *Data Catalog Vocabulary (DCAT) – Version 2*, W3C Recommendation, February 2020. https://www.w3.org/TR/vocab-dcat-2/

[3] W3C, *ODRL Information Model 2.2*, W3C Recommendation, February 2018. https://www.w3.org/TR/odrl-model/

[4] H. Krawczyk, M. Bellare, R. Canetti, *HMAC: Keyed-Hashing for Message Authentication*, RFC 2104, IETF, 1997.

[5] C. Dwork, A. Roth, *The Algorithmic Foundations of Differential Privacy*, Foundations and Trends in Theoretical Computer Science, 9(3–4), 2014.

[6] L. Sweeney, *k-anonymity: a model for protecting privacy*, International Journal of Uncertainty, Fuzziness and Knowledge-Based Systems, 10(5), 557–570, 2002.

[7] FastAPI documentation. https://fastapi.tiangolo.com/

[8] SQLite documentation. https://www.sqlite.org/docs.html